

AI log Part A

AI Log - Task 1

AI assistance was used to clarify the expected scope of Task 1 and to identify suitable tools for APK decompilation. Based on the AI guidance, I used JADX/JADX-GUI to open and inspect the provided APK.

The AI helped me understand which files were relevant to the assignment requirements, including `AndroidManifest.xml` and `MainActivity.java`. After reviewing the decompiled output, I identified the package structure, the main activity, and the WebView-related code.

AI assistance was also used to interpret the security relevance of the findings. The key evidence identified included `android:usesCleartextTraffic="true"` in `AndroidManifest.xml`, `webView.loadUrl("http://www.example.com")` in `MainActivity.java`, and `sslErrorHandler.proceed()` inside the WebView SSL error handler. These findings were then used as supporting evidence for the later insecure network transmission analysis.

AI Log - Task 2

AI was used to assist in understanding the decompiled Android application and constructing a network-focused threat model. It helped structure the analysis by identifying key aspects such as network communication behaviour, potential data transmitted over the network, and the components of a basic system and threat model.

From the decompiled code, the application was identified as initializing a WebView and loading `http://www.example.com` on startup, indicating that it functions as a simple web content viewer that relies on remote resources.

The analysis included possible network data such as requested URLs, HTTP headers, server responses (e.g., HTML and JavaScript), and potential user or session-related data depending on web content.

A basic threat model was derived consisting of the application, a remote server, an untrusted network, and an on-path attacker such as a user on the same public Wi-Fi network.

AI Log - Task 3

Prompt summary: We asked the AI how to interpret Part A Task 3 and what type of network vulnerability we should focus on.

Response summary: The AI explained that Task 3 mainly focuses on insecure transport / TLS misconfiguration issues, such as cleartext HTTP traffic, allowing cleartext traffic, ignoring SSL certificate errors, or bypassing hostname verification. These issues could enable man-in-the-middle (MITM) attacks.

Validation: We checked the assignment specification and rubric and confirmed that Task 3 explicitly requires static analysis of insecure transport paths, such as cleartext traffic or disabled certificate/hostname checks. Therefore, we focused our analysis on Android network configuration, WebView loading behavior, and TLS-related code.

Prompt summary: We provided the AI with our initial findings, including `android:usesCleartextTraffic="true"` in `AndroidManifest.xml` and `webView.loadUrl("http://www.example.com")` in `MainActivity.java`, and asked it to help draft the "Insecure Transport Path" section.

Response summary: The AI summarized that the APK allows plaintext HTTP traffic and actually loads an HTTP resource through a WebView. This means encrypted transport is not enforced for that communication path, which could expose the traffic to MITM interception or tampering.

Validation: We manually checked the decompiled code and confirmed that `AndroidManifest.xml` contains `android:usesCleartextTraffic="true"` and that `MainActivity.java` calls `webView.loadUrl("http://www.example.com")`. This shows that the issue exists both in configuration and in actual code behavior.

Prompt summary: After another group member found SSL handling code, we asked the AI whether this evidence should be included in Task 3 and how it relates to MITM risk.

Response summary: The AI explained that calling `sslErrorHandler.proceed()` inside `onReceivedSslError()` is insecure because the app continues loading the page even when an SSL certificate error occurs. It also explained that a `HostnameVerifier.verify()` method returning `true` weakens TLS identity verification because it accepts any hostname as valid.

Validation: We manually confirmed that `MainActivity.java` contains `sslErrorHandler.proceed()`; and that `HostnameVerifier.verify(...)` returns `true`; Based on this evidence, we expanded the report from a cleartext HTTP issue to a broader insecure transport / TLS verification issue.

Prompt summary: We asked the AI to explain, in plain language, what a man-in-the-middle attacker could do with this vulnerability and why normal HTTPS prevents this type of attack.

Response summary: The AI explained that on an untrusted network, such as public Wi-Fi, an attacker could read HTTP traffic, modify page content, insert fake prompts, alter links, inject scripts, or redirect users. It also explained that proper HTTPS helps prevent this by encrypting traffic and validating the server certificate and hostname.

Validation: We did not claim that we stole credentials or attacked real users. Instead, we used the code evidence to describe a realistic impact: because the app allows HTTP traffic and weakens SSL/hostname validation, an on-path attacker may be able to read or modify WebView-loaded content. This impact is consistent with the static analysis evidence.

Prompt summary: We asked the AI to help turn the Task 3 finding into concise wording for our report and presentation.

Response summary: The AI suggested a slide structure and a short English presentation script covering the insecure transport path, code/configuration evidence, SSL error handling, and MITM impact.

Validation: We treated the AI output as a draft and manually edited it to fit the two-page report limit and presentation time limit. Every final claim was checked against code evidence, including `android:usesCleartextTraffic="true"`, `webView.loadUrl("http://www.example.com")`, `sslErrorHandler.proceed()`, and `HostnameVerifier.verify()` returning true.

Prompt summary: We asked the AI how to start Part A Task 3 from the provided APK file and whether we needed to unpack and decompile the APK first.

Response summary: The AI explained that the first step should be to unpack and decompile the APK using apktool and jadx. It clarified that apktool is useful for inspecting `AndroidManifest.xml`, resources, and smali code, while jadx is useful for reading decompiled Java source code.

Validation: We ran apktool and jadx locally on the provided APK. apktool successfully produced the unpacked APK directory, and jadx produced a decompiled source directory even though it reported some decompilation errors. We manually opened the generated folders and confirmed that `AndroidManifest.xml` and `MainActivity.java` were available for analysis.

Prompt summary: We asked the AI how to open and inspect the decompiled APK files in VS Code.

Response summary: The AI explained that the decompiled folders could be opened directly in VS Code without compiling them. It pointed us to the jadx source folder for Java code and the apktool folder for manifest, resources, and smali output.

Validation: We opened the jadx output folder in VS Code and manually navigated to `sources/com/example/mastg_test0019/MainActivity.java`. We also opened `AndroidManifest.xml` from the resources/apktool output. This allowed us to visually confirm the relevant files and line locations used in the report.

Prompt summary: We asked the AI whether reviewing AndroidManifest.xml, Java code, and smali code without running the app counts as static analysis.

Response summary: The AI confirmed that this is static analysis because the app was not executed. It explained that static analysis includes inspecting decompiled source code, manifest configuration, resources, and bytecode/smali to identify security issues.

Validation: We confirmed that our Part A evidence came from decompiled files rather than runtime testing. The code/configuration evidence was collected from AndroidManifest.xml, MainActivity.java, and smali files, so we described this part of the methodology as static analysis in the report.

AI Log - Task 4

Prompt summary: We asked the AI how to approach mitigation design after identifying an insecure vulnerability in the APK.

Response summary: The AI suggested that the mitigation should focus on removing the insecure communication path rather than only describing the risk. It guided us to think about the issue from three layers: fixing the direct insecure endpoint, enforcing secure traffic at the Android configuration level, and adding a more specific network security rule as defense-in-depth.

Validation: We compared this approach with the evidence from previous tasks. Since the vulnerability involved cleartext HTTP traffic and weak transport security, the mitigation ideas needed to directly reduce MITM opportunities. We therefore selected HTTPS enforcement, cleartext traffic restriction, and Network Security Configuration as the main mitigation directions.

Prompt summary: We asked the AI how to evaluate whether the proposed mitigations were strong enough.

Response summary: The AI explained that good mitigations should address both the root cause and possible future mistakes. For example, changing HTTP to HTTPS fixes the immediate insecure connection, while disabling cleartext traffic prevents similar insecure connections from being introduced later.

Validation: We checked that each mitigation had a clear security purpose. HTTPS protects confidentiality and server authentication, the manifest setting acts as a general safeguard, and Network Security Configuration provides domain-specific control. This confirmed that the final mitigation strategy followed a layered defense model.

Prompt summary: We asked the AI how to make the mitigation plan more complete instead of relying on only one fix.

Response summary: The AI explained that a single fix may reduce the immediate issue, but a stronger solution should include backup controls. It suggested thinking about the mitigation as defense-in-depth, where the code-level fix, manifest-level restriction, and network security configuration work together.

Validation: We used this idea to make sure the mitigation plan was not limited to changing the URL from HTTP to HTTPS. We included additional Android security controls so that insecure cleartext traffic would still be blocked even if a developer accidentally introduced another HTTP endpoint later.

Prompt summary: We asked the AI how to explain the security value of the mitigations without adding unsupported claims.

Response summary: The AI advised focusing on what the mitigations realistically prevent: unencrypted traffic exposure, traffic modification, and malicious WebView content injection. It also reminded us not to claim that the fixes completely eliminate all possible network attacks.

Validation: We kept the final explanation focused on reducing MITM-related risks. We described the mitigations as improving confidentiality, server authentication, and traffic integrity, while avoiding overbroad claims that the app would become fully secure against every type of attack.

Prompt summary: We asked the AI how to connect the mitigations to risk reduction.

Response summary: The AI helped us reason that the mitigations reduce the attacker's ability to intercept, read, or modify WebView traffic. It also clarified that the main security improvement is reducing the likelihood of MITM attacks and malicious content injection.

Validation: We ensured that the final reasoning stayed within the scope of the evidence. We did not claim that the vulnerability was dynamically exploited against real users. Instead, we explained how the proposed mitigations would reduce the risks identified through static analysis.

AI log Part B

Task1:

Prompt summary: I asked the AI to help me understand which bug bounty target would be suitable for Part B and how to stay within the legal scope of the program.

Response summary: The AI helped me compare different programs and guided me toward the Via / Citymapper Android app because it was explicitly listed as an in-scope mobile application. The AI also reminded me to use only my own account, avoid accessing third-party data, and keep testing within the published scope.

Validation: I checked the Via program scope manually and confirmed that the Citymapper Android app package `com.citymapper.app.release` was listed as in scope. I also confirmed that `citymapper.com/*` and `*.citymapper.com` were out of scope, so I limited the finding to Android app behavior rather than testing the public website.

Prompt summary: I asked the AI to help me analyze the Citymapper Android APK and identify code paths related to magic-link login handling.

Response summary: The AI helped locate relevant decompiled files and explained that the magic-link flow involved `URLHandlerActivity` and `MagiclinkLoginActivity`. It identified that incoming deep links were handled by the Android app and that the login token was carried inside the `access_token` query parameter.

Validation: I manually checked the decompiled APK files and confirmed the relevant code paths. In `AndroidManifest.xml`, `URLHandlerActivity` is exported and accepts external VIEW intents. In `URLHandlerActivity.java`, the incoming URI is processed before being routed to the magic-link login flow.

Prompt summary: I asked the AI to help identify the specific static-analysis evidence showing that a sensitive magic-link token could be exposed.

Response summary: The AI identified that `URLHandlerActivity.java` stores the full incoming URI using `uri.toString()` and passes it to the `HANDLE_CM_URI` logging flow. It also explained that this happens before the app checks whether the URI is a magic-link login request.

Validation: I verified the code manually. In `URLHandlerActivity.java`, the app places the full URI into the `"Uri"` field and calls `com.citymapper.app.common.util.n.f("HANDLE_CM_URI", map, map2)`. I also confirmed that the magic-link check for `auth_provider=magiclink` happens after this logging call.

Prompt summary: I asked the AI to help confirm whether the exposed URI parameter was actually security-sensitive.

Response summary: The AI explained that the `access_token` parameter is security-sensitive because it is later extracted from the deep link and used to authenticate the user through the magic-link login flow.

Validation: I checked the decompiled code and confirmed that `MagiclinkLoginActivity` reads `data2.getQueryParameter("access_token")` and uses it to create an `AuthRequest` with `AuthProvider.MAGICLINK`. This confirmed that the token in the URL is not just metadata, but an authentication credential.

Prompt summary: I asked the AI to help design a safe proof of concept for demonstrating the issue without exposing real user data.

Response summary: The AI suggested a PoC process using only my own Citymapper account. The process involved generating a magic-link login email, opening the link in the Android app, collecting redacted Android activity/log evidence, extracting the token-bearing link, and replaying it to show that the exposed link could still be used for login.

Validation: I tested this on an Android emulator with my own account. I redacted the real `access_token` in saved evidence files and avoided submitting raw token values. The PoC evidence showed that the activity dump contained a redacted token-bearing login URI and that replaying the extracted link opened `MagiclinkLoginActivity` and resulted in a successful login screen.

Prompt summary: I asked the AI to help me prepare a response to the platform after they requested stronger proof of actual token exposure and successful token use.

Response summary: The AI helped me organize the evidence around two points: actual exposure of the `access_token` in accessible Android diagnostic output, and successful use of the extracted token-bearing link after extraction. It also helped me phrase the response carefully without overstating impact.

Validation: I checked the generated evidence before using it. The key files included redacted activity dump output, a redacted extracted-link file, a SHA-256 hash of the extracted link, replay command output, and a screenshot showing successful login. I made sure the final text did not contain the raw token.

Prompt summary: I asked the AI to help prepare the Part B report and presentation content, especially the target description and explanation of where the issue was found.

Response summary: The AI helped summarize the target as the official Citymapper Android app, package com.citymapper.app.release, tested from Google Play, and within Via's HackerOne scope. It also helped explain that the issue was found in the Android magic-link login flow involving URLHandlerActivity and MagiclinkLoginActivity.

Validation: I cross-checked the target details against the program scope and the installed app package information. I confirmed the package name, version, and installer source from Android package information, and I confirmed that the testing used my own account only.

Task2:

Prompt summary: I asked the AI to help me analyse a potential vulnerability I found while completing a cybersecurity coursework assignment. I explained that I had installed the APK on an Android emulator, decompiled it in VS Code, and noticed a possible "Magic Link Token Logging" issue. Specifically, I found that URLHandlerActivity.java appeared to log the full uri.toString() into a "Uri" field before checking whether the same deep link was a magic-link login request, and that other files appeared to read access_token from the same deep link for AuthProvider.MAGICLINK authentication.

Response summary: The AI helped me understand that this could be a sensitive token exposure issue if a magic-link access_token is included in the deep link and the full URI is logged before authentication handling. The AI guided me through the relevant code chain, including URLHandlerActivity.java, mx/t.java, m5/e.java, n.java, mp/b.java, pe/o.java, and kf/e.java. It also explained the difference between static evidence, local logging evidence, authentication-flow evidence, and actual production token redemption, so I would not overclaim the impact.

Validation: I manually reviewed the decompiled source files and confirmed that the Android app logs the incoming deep link URI and later reads access_token from a magic-link deep link for authentication. I also confirmed that the finding should be limited to the Android app asset com.citymapper.app.release rather than out-of-scope Citymapper web domains.

Prompt summary: I asked the AI whether emulator testing or online testing could be considered an attack, and how to stay within responsible testing boundaries.

Response summary: The AI advised me to start with static analysis and offline emulator testing, avoid using real user accounts or real tokens at first, avoid

out-of-scope domains, and avoid any action that could access third-party data. It explained that fake-token testing in an offline emulator is much lower risk than real production token redemption.

Validation: I began with offline tests only. I used fake `access_token` values and did not attempt to log in to a real account or contact production authentication services during the initial proof of concept.

Prompt summary: I asked the AI to help me identify which HackerOne scope or asset this issue would belong to.

Response summary: The AI guided me to select the Via / Citymapper Android app asset `com.citymapper.app.release` because it was listed as an in-scope Android Play Store asset. It also warned me that `citymapper.com` and `*.citymapper.com` were listed as out of scope, so I should not test or claim impact against those web assets.

Validation: I manually checked the program scope and confirmed that `com.citymapper.app.release` was in scope, while `citymapper.com` and `*.citymapper.com` were out of scope. I limited the finding to Android app behavior.

Prompt summary: I asked the AI to help me build a safe proof of concept for the suspected token logging issue.

Response summary: The AI guided me through an offline fake-token emulator test using a `citymapper:// magic-link` deep link containing a fake `access_token`. It helped me confirm that the app opened the magic-link login activity and explained that this showed the deep-link path was reachable without using a real token.

Validation: I ran the fake-token deep link on the emulator and observed that the app opened `MagiclinkLoginActivity`. This supported the hypothesis that a token-like value in the deep link reaches the magic-link handling path.

Prompt summary: After via asked for proof of actual token exposure in logs, I asked the AI how to safely produce stronger evidence.

Response summary: The AI explained that the official Play Store build is not debuggable, so app-private event logs are not normally readable. It guided me through creating a local re-signed/debuggable test build for controlled offline analysis, adding local instrumentation, and inspecting the app's local event storage under `no_backup/cm_events`.

Validation: In the offline instrumented test build, I confirmed that a fake `access_token` appeared in the app's local event logging pipeline and local event storage. This supported the claim that a token-like value can be written into local app event logs.

Prompt summary: I asked the AI whether the token was only logged or whether it was also used by the app's magic-link authentication flow.

Response summary: The AI guided me through a second offline instrumentation test. It helped add a local PoC log to the AuthRequest constructor when AuthProvider.MAGICLINK was used, so I could check whether the same fake access_token value entered the authentication request object.

Validation: The offline test showed the same fake access_token appearing in the logging/event pipeline and later in AuthRequest(AuthProvider.MAGICLINK, accessToken). This proved the client-side chain from deep link access_token to logging/event pipeline to magic-link authentication request.

Prompt summary: I asked the AI to help generate a HackerOne-style vulnerability report for submit to check it.

Response summary: The AI helped draft a structured report including the vulnerability title, affected asset, weakness category, affected version, technical evidence, offline PoC steps, expected result, actual result, impact, severity reasoning, scope limitations, remediation suggestions, and a responsible testing statement. It also helped me phrase the report conservatively so that I did not claim confirmed account takeover or unauthorized access to production logs.

Validation: I reviewed the generated report manually and used it as the basis for my coursework submission. I confirmed that no full real token, real magic-link URL, or third-party account information was included.

Prompt summary: I asked the AI how to answer follow-up questions, which required proof of actual access-token exposure in accessible logs and successful use of the exposed token after extraction.

Response summary: The AI helped me separate the evidence into levels. It explained that my offline tests could prove local event-log exposure and client-side authentication-flow use, but not necessarily external attacker access or production token redemption. It proposed a risk-based testing plan, starting with static and offline tests and only moving to a real own-account validation if explicitly authorized.

Validation: I used this framework to avoid overclaiming. I described the offline proof as evidence that a token-like value enters local app logging and the magic-link AuthRequest, while reserving production redemption claims for an authorized own-account test.

Prompt summary: After receiving permission to test only my own account, I asked the AI to guide me through a minimal real-account magic-link validation.

Response summary: The AI instructed me to use a disposable test email, avoid adding personal data, avoid binding payment information, avoid sharing the full magic-link URL or full access_token, and record only a redacted token identifier such as the final six characters. It then guided me through logging out, requesting a new magic-link email, copying the link locally, recording only a token suffix, and opening the link to verify login.

Validation: I used only my own test account. I requested a real Citymapper magic-link email, recorded only a redacted token suffix, opened the link, and confirmed that the app displayed a successful secure sign-in message and returned to the same test account. I did not access any third-party account or data.

Prompt summary: I asked the AI to help me understand the limits of the evidence and how to present it accurately.

Response summary: The AI explained that the evidence supports a client-side risk: a magic-link access_token can enter the app logging/event pipeline and the same token can be used by the magic-link authentication flow. It also explained that I should not claim that an attacker definitely has access to Citymapper's backend logs or third-party analytics logs, because that was not proven.

Validation: In my final write-up, I limited the conclusion to the evidence I actually collected: static code evidence, offline fake-token logging/auth-flow proof, and an authorized own-account magic-link login validation. I did not claim confirmed compromise of other users or unauthorized access to production logs.

Task3:

I used AI to clarify my Part B role and draft the impact section. AI helped me explain that the exposed magic-link `access\_token` could allow replay and account/session compromise, while keeping the claim accurate by distinguishing rooted replay evidence from no-root diagnostic exposure.

Task4:

Prompt summary: I asked the AI how to structure the Impact Justification section for the Citymapper magic-link token exposure vulnerability.

Response summary: The AI suggested organizing the section into three parts: affected parties, exposed information, and potential consequences. It explained that this structure would make the impact easier to present and would clearly show who is affected, what information is exposed, and what risks may result.

Validation: I used this structure in my final slides. The first slide covers affected parties and exposed information, while the second slide covers potential consequences.

Prompt summary: I asked the AI how to decide whether logs and diagnostic output should be described as victims or exposure channels.

Response summary: The AI explained that logs, Activity state, and diagnostic output are not affected parties. They are exposure surfaces where sensitive information may appear. The affected parties are the users, their accounts, and the service provider.

Validation: I revised the structure so that users and Citymapper were listed as affected parties, while Activity state, diagnostic output, and logs were listed as exposure surfaces.

Prompt summary: I asked the AI to explain the difference between exposed information and exposure surfaces.

Response summary: The AI explained that the full magic-link URL containing access_token is the sensitive information being exposed, while Activity state, diagnostic output, and logs are the surfaces where this information may appear.

Validation: I updated the slide to separate these ideas clearly. The slide lists the full magic-link URL and deep-link login parameters as exposed information, and lists Activity state, diagnostic output, and logs as exposure surfaces.

Prompt summary: I asked the AI how to think about the impact of a vulnerability from a security-analysis perspective.

Response summary: The AI suggested that impact should not only describe the technical bug itself, but should connect the bug to affected users, exposed assets, attacker opportunities, and possible business or privacy consequences.

Validation: I used this reasoning to organize my impact analysis around who is affected, what sensitive information is involved, and what could happen if the information is misused.

Prompt summary: I asked the AI how to phrase the impact without overstating the evidence.

Response summary: The AI advised using cautious wording such as “may,” “risk,” and “potential” because the evidence supports token exposure and possible replay risk, but does not prove compromise of other users or unauthorized access to backend systems.

Validation: I revised the slide wording to avoid absolute claims. For example, I used “Account takeover risk” and “An attacker may replay a valid token” instead of claiming that account takeover definitely occurred.

Prompt summary: I asked the AI to check whether my Impact Justification slides were logically clear and accurate.

Response summary: The AI confirmed that the slides were structured clearly but suggested improving some wording. It recommended changing the affected parties wording to “Citymapper Android users who log in via magic link, and their accounts” and using “Exposure surfaces: Activity state, diagnostic output, and logs” to avoid repetition.

Validation: I applied these wording changes in the final slides. The revised version is clearer because it separates who is affected, what is exposed, where it appears, and what consequences may follow.

Task5:

I used AI to analyze a Citymapper Android security issue involving magic-link login URLs. The main finding was that the official Android app may expose token-bearing login URLs in diagnostic surfaces before properly removing sensitive data.

Besides, I also used AI to summarize the target app, the vulnerability, the possible impact, and why the issue matters. AI also helped explain that a magic-link URL works like a temporary login key, and if its access_token is exposed, it may be replayed to log in.